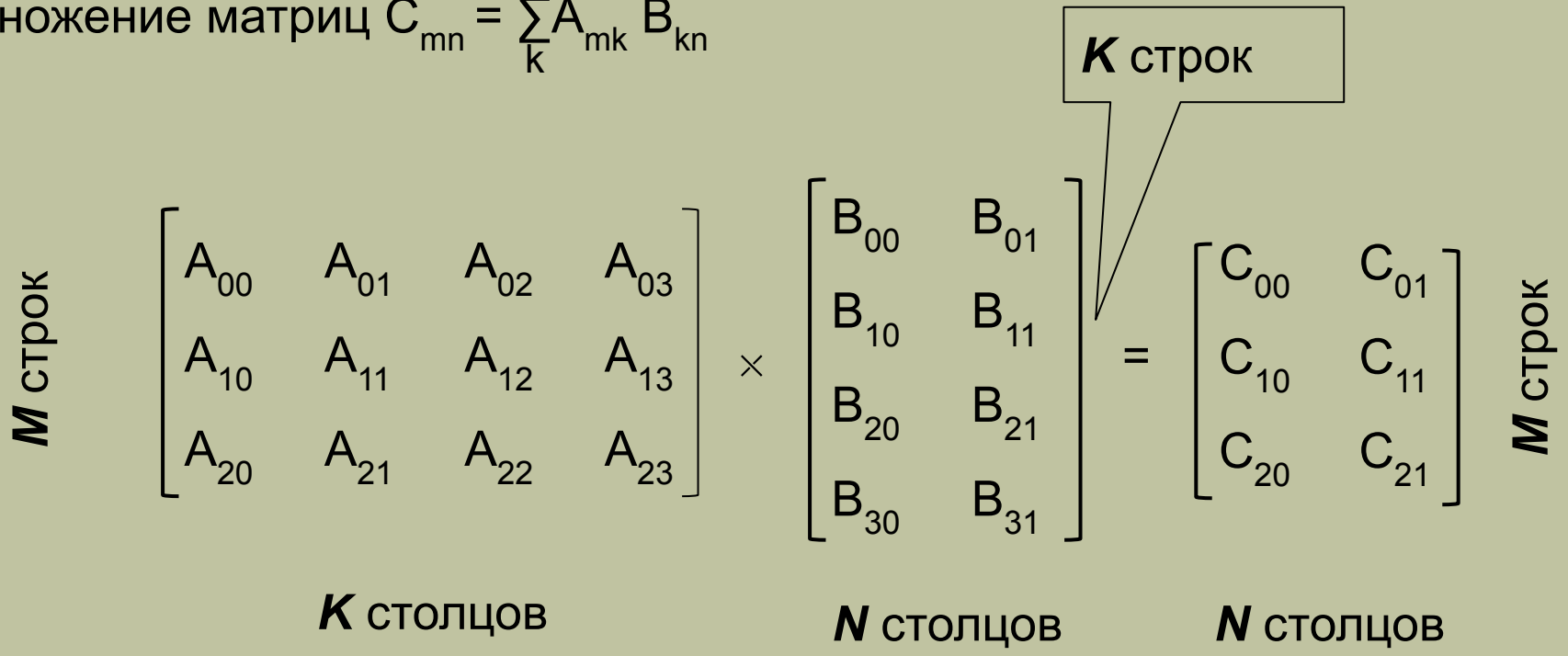
Лекция 13

- Произведение матриц.
- Реализация произведения матриц на основе CUDA API.
- Реализация произведения матриц на основе numba.

Произведение матриц

Умножение матриц $C_{mn} = \sum_k A_{mk} B_{kn}$



```
#include <malloc.h>
#include <ctime>
#define M 1024
#define K 1024
#define N 1024

void hMultMats(double** A, double** B, double** C){
  double acc;
  for(int m=0; m<M;m++)
    for(int n=0; n<N;n++){
      acc=0.0;
      for(int k=0; k<K;k++)
        acc+=A[m][k]*B[k][n];
      C[m][n]=acc;
    }
}
```

```
void hInitMats(double** A, double** B){
  for(int m=0; m<M;m++)
     for(int k=0; k<K; k++)
        A[m][k]=(double)((k+m*K)*1.0E-5);

  for(int k=0; k<K;k++)
     for(int n=0; n<N;n++)
        B[k][n]=1.0;
}
```

```
Int main(){
 double **A, **B, **C;
  A=(double**)calloc(M, sizeof(double*));
  for(int m=0; m<M; m++)
    A[m]=(double*)calloc(K, sizeof(double));
……
hInitMats(A, B);
clock_t start=clock();
  hMultMats(A, B, C);
clock_t finish=clock();
……..
  hMatOut(A, M, K);
  hMatOut(B, K, N);
  hMatOut(C, M, N);
……
}
```

```
/Lecture8/Lab8> ./lab8cpu
0       0.00128 0.00256 0.00384 0.00512 0.0064  0.00768 0.00896
1.31072 1.312   1.31328 1.31456 1.31584 1.31712 1.3184  1.31968
……..
9.17504 9.17632 9.1776  9.17888 9.18016 9.18144 9.18272 9.184
////////////////////////////////////////////////////////////
1       1       1       1       1       1       1       1
1       1       1       1       1       1       1       1
……..
////////////////////////////////////////////////////////////
5.23776 5.23776 5.23776 5.23776 5.23776 5.23776 5.23776 5.23776
1347.42 1347.42 1347.42 1347.42 1347.42 1347.42 1347.42 1347.42
2689.59 2689.59 2689.59 2689.59 2689.59 2689.59 2689.59 2689.59
……..
9400.48 9400.48 9400.48 9400.48 9400.48 9400.48 9400.48 9400.48
Elapsed time: 4173.59 ms
```

```
/Lecture8/Lab8> g++  lab8cpu.cpp  -pg -o lab8cpu
/Lecture8/Lab8> ./lab8cpu
/Lecture8/Lab8> ls -ltr
итого 720
-rw-r--r-- 1 malkov users   1528 Mar 16 18:27 lab8cpu.cpp
-rwxr-xr-x 1 malkov users  20496 Mar 16 19:22 lab8cpu
-rw-r--r-- 1 malkov users   1023 Mar 16 19:22 gmon.out
/Lecture8/Lab8> gprof lab8cpu gmon.out > lab8cpu.prof
```

```
/Lecture8/Lab8> vim lab8cpu.prof
Flat profile:

Each sample counts as 0.01 seconds.
 %   cumulative  self    self    total
time      secs      secs  calls   s/call    s/call            name
100.40  3.98     3.98      1       3.98     3.98  hMultMats(double**, double**, double**)
 0.25     3.99      0.01    1        0.01     0.01  hInitMats(double**, double**)
.....
```

```
#include <malloc.h>
#define M 1024
#define K 1024
#define N 1024
#define BLOCK_DIM 32

__global__ void gMultMats(float* A, float* B, float* C){
  int n=threadIdx.x + blockIdx.x*blockDim.x;
  int m=threadIdx.y + blockIdx.y*blockDim.y;

  float acc=0.0;
  for(int k=0; k<K; k++)
    acc+=A[k+m*K]*B[n+k*N];
  C[n+m*N]=acc;
}
```

```
__global__ void gInit(float* D, int s){
  int j=threadIdx.x + blockIdx.x*blockDim.x;
  int i=threadIdx.y + blockIdx.y*blockDim.y;
  int J=blockDim.x*gridDim.x;

  D[j+i*J]=s*(float)((j+i*J)*1.0E-5)+(1-s)*1.0f;
}
```

```
int main(){
  float *A, *B, *C;
  cudaMalloc((void**)&A, M*K*sizeof(float));
……
  gInit<<<dim3(K/32, M/32),dim3(32,32)>>>(A,1);
  cudaDeviceSynchronize();
  gInit<<<dim3(N/32, K/32),dim3(32,32)>>>(B,0);
  cudaDeviceSynchronize();
  cudaMemset(C, 0, M*N*sizeof(REAL));

  gMultMats<<<dim3(N/BLOCK_DIM, M/BLOCK_DIM),
                    dim3(BLOCK_DIM, BLOCK_DIM)>>>(A,B,C);
  cudaDeviceSynchronize();

  hMatOut(A, M, K);
……..
```

```
/Lecture8/Lab8> ./lab8gpu
0       0.00128 0.00256 0.00384 0.00512 0.0064  0.00768 0.00896
1.31072 1.312   1.31328 1.31456 1.31584 1.31712 1.3184  1.31968
……
9.17504 9.17632 9.1776  9.17888 9.18016 9.18144 9.18272 9.184
////////////////////////////////////////////////////////////
1       1       1       1       1       1       1       1
1       1       1       1       1       1       1       1
……..
////////////////////////////////////////////////////////////
5.23776 5.23776 5.23776 5.23776 5.23776 5.23776 5.23776 5.23776
1347.42 1347.42 1347.42 1347.42 1347.42 1347.42 1347.42 1347.42
2689.59 2689.59 2689.59 2689.59 2689.59 2689.59 2689.59 2689.59
……..
9400.48 9400.48 9400.48 9400.48 9400.48 9400.48 9400.48 9400.48
```

```
/Lecture8/Lab8> nvprof ./lab8gpu

    Type
  Time(%)  Time     Calls   Avg         Min         Max              Name
GPU activities:
63.25%  3.9720ms   1  3.9720ms  3.9720ms  3.9720ms  gMultMats(float*, float*, float*)
 1.90%   119.46us  2  59.728us  59.648us  59.808us  gInit(float*, int)
```

3.98s*1000/3.9720=**1002.0141**

Ускорение в тысячу раз!😲

```
import numpy as np
from numba import jit
from numba import cuda
from time import  perf_counter_ns  as timer
from numba import cuda, float32

@cuda.jit#(argtypes=[float32[:,:], float32[:,:], float32[:,:]])
def matmul(A, B, C):
   m,n = cuda.grid(2)
   if m < C.shape[0] and n < C.shape[1]:
       acc = 0.
       for k in range(A.shape[1]):
           acc += A[m, k] * B[k, n]
       C[m, n] = acc
```

```
M=1024
K=1024
N=1024

A=np.arange(M*K)
A=A.reshape(M,K)
B=np.ones((K,N))
#C=np.zeros((M,N))
np.set_printoptions(formatter={'float': '{: 0.3g}'.format})

start = timer()
C=np.matmul(A,B)
dt = timer() - start
dt=dt/1000000.0
print ("np.matmul time %f ms" % dt)
print(C)
```

```
A=np.float32(A)
B=np.float32(B)
C=np.float32(C)

dA=cuda.to_device(A)
dB=cuda.to_device(B)
dC=cuda.to_device(C)
```

```
start = timer()
matmul[(64,64),(16,16)](dA,dB,dC)
cuda.synchronize()
dt = timer() - start

#dC.to_host()
dt=dt/1000000.0
print ("matmul time %f ms" % dt)
print(dC)
```

```
/Lab10> python cuda-gemm3.py
np.matmul time 20.236182 ms
[[ 5.24e+05  5.24e+05  5.24e+05 ...  5.24e+05  5.24e+05  5.24e+05]
 [ 1.57e+06  1.57e+06  1.57e+06 ...  1.57e+06  1.57e+06  1.57e+06]
 ...
 [ 1.07e+09  1.07e+09  1.07e+09 ...  1.07e+09  1.07e+09  1.07e+09]]
matmul time 30.242027 ms
[[ 5.24e+05  5.24e+05  5.24e+05 ...  5.24e+05  5.24e+05  5.24e+05]
 [ 1.57e+06  1.57e+06  1.57e+06 ...  1.57e+06  1.57e+06  1.57e+06]
 ...
 [ 1.07e+09  1.07e+09  1.07e+09 ...  1.07e+09  1.07e+09  1.07e+09]]
```

```
/Lab10> nvprof python cuda-gemm3.py
```

```
Type
Time(%)   Time      Calls       Avg             Min          Max          Name
GPU activities:
49.23%  62.324ms   2       31.162ms    30.458ms    31.866ms
cudapy::__main__::matmul$242(Array<float, int=2, A, mutable, aligned>,
Array<float, int=2, A, mutable, aligned>, Array<float, int=2, A, mutable, aligned>)
```